Sprint 4 – Week 3 Closeout Report

SWENG 894 Capstone

MicroMatch

GitHub Repository Link

Chastidy Joanem

1. System Overview and Context

1.1 System Scope

MicroMatch is a web-based micro-internship marketplace that connects students with short-term, skill-based professional opportunities. The platform currently supports the full project lifecycle from account creation through project completion.

The system architecture continues to follow a layered full-stack design:

- **Frontend:** React-based SPA (Single Page Application)
- **Backend:** FastAPI REST API
- **Database:** PostgreSQL (relational persistence)
- **Authentication:** JWT-based stateless authentication
- **Authorization:** Role-Based Access Control (RBAC)

Primary user roles:

- Student
- Organization
- Administrator

1.2 Sprint 1 – 3 Recap

Sprint 1 delivered the Minimum Viable Marketplace (MVP Foundation), including user authentication and registration (JWT), role-based access control (Student / Organization), student and organization profile management, project posting, and basic project browsing and search.

Sprint 2 introduced the Application Workflow Layer, enabling students to apply to projects, track application status, and allowing organizations to accept or reject applications (US-1 through US-9, FR-1 through FR-9).

Sprint 3 completed the core system lifecycle by implementing deliverable submission (FR-10), deliverable review (FR-11), project completion (FR-12), the feedback system (FR-13), admin moderation (FR-14), and system-wide logging middleware (FR-15). At the conclusion of Sprint 3, all foundational functional requirements had been satisfied and the platform supported a full end-to-end micro-internship lifecycle.

1.3 Sprint 4 Roadmap Context

Sprint 4 focuses on transforming MicroMatch from a workflow-complete marketplace into an intelligent, engagement-driven platform. The sprint introduces four major feature areas that were identified during earlier backlog refinement:

Planned Sprint 4 enhancements:

- Notifications system – FR-16 / US-16
- Messaging system – FR-17 / US-17
- Dashboard Analytics – FR-18 / US-18
- Profile enhancements – FR-19 / US-19
- Smart Matching & Ranking Algorithm – SA-1

1.4 Functional Requirements to User Stories Mapping

Functional Requirement	User Story
FR-1: The system shall allow users to create an account.	US-1
FR-2: The system shall support role-based access control.	US-1
FR-3: The system shall allow students to create/edit profile.	US-2
FR-4: The system shall allow organizations to create profile.	US-3
FR-5: The system shall allow orgs to post listings.	US-4
FR-6: The system shall allow students to browse/search listings.	US-5
FR-7: The system shall allow students to apply.	US-6
FR-8: The system shall track application status.	US-8
FR-9: The system shall allow orgs to accept/reject.	US-9
FR-10: The system shall allow students to submit deliverables.	US-10

FR-11: The system shall allow orgs to review deliverables.	US-11
FR-12: The system shall allow marking projects complete.	US-12
FR-13: The system shall allow feedback submission.	US-13
FR-14: The system shall allow admin moderation.	US-14
FR-15: The system shall log system actions.	US-15
FR-16: The system shall notify users of key events (application decisions, deliverable reviews, project completion).	US-16
FR-17: The system shall allow users to send/receive messages within a project.	US-17
FR-18: The system shall provide users with basic analytics (applications, completed projects).	US-18
FR-19: The system shall allow profiles to display skills, completed projects, and ratings.	US-19
SA-1: Smart Matching & Ranking Algorithm.	US-20

This traceability ensures alignment between formal IEEE-style functional requirements and Agile user stories used for sprint planning.

2. Sprint 4 – Week 3 Overview

2.1 Sprint Goal

The overarching goal of Sprint 4 is:

- To transform MicroMatch from a workflow-complete marketplace into an intelligent engagement-driven platform through notifications, smart recommendations, messaging, analytics, and profile enhancements.

The goal of Sprint 4 – Week 3 is:

- To implement the messaging system, dashboard analytics, and profile enhancements – completing the Sprint 4 scope and finalizing the MicroMatch MVP.

3. Sprint Planning

3.1 Groomed Product Backlog

Priority Definitions

- Highest: Critical sprint milestone with significant algorithm or core infrastructure.
- High: Core MVP functionality required for end-to-end workflow.
- Medium: Important workflow enhancements dependent on core features.
- Low: Administrative or cross-cutting features not required for initial MVP validation.

ID	User Story	Priority	Story Points	Sprint	Rationale
US-1	Registration/Login	High	5	Sprint 1	Required for all access
US-2	Student Profile	High	5	Sprint 1	Core student data
US-3	Org Profile	High	5	Sprint 1	Required before posting
US-4	Post Listing	High	8	Sprint 1	Core marketplace feature
US-5	Browse/Search	High	5	Sprint 1	Enables discovery
US-6	Apply	High	5	Sprint 2	Core interaction
US-7	Review Applications	Medium	5	Sprint 2	Depends on applications
US-8	Track Status	High	3	Sprint 2	Enhances UX
US-9	Accept/Reject	Medium	3	Sprint 2	Workflow control
US-10	Submit Deliverables	Medium	5	Sprint 3	Completion flow
US-11	Review Deliverables	Medium	3	Sprint 3	Completion validation
US-12	Mark Complete	Medium	3	Sprint 3	Project closure

US-13	Feedback	Low	5	Sprint 3	Post-completion feature
US-14	Admin Moderation	Low	5	Sprint 3	Governance feature
US-15	Logging	Low	8	Sprint 3	Cross-cutting concern
US-16	Notifications	Medium	5	Sprint 4	User awareness
US-17	Messaging	Medium	8	Sprint 4	Communication
US-18	Analytics	Low	5	Sprint 4	Insights
US-19	Profile Enhancements	Low	3	Sprint 4	UX Improvements
US-20 / SA-1	Smart Matching Algorithm	Highest	8	Sprint 4	Significant Algorithm

3.2 Sprint Backlog – Week 3 Commitment

Based on historical velocity and current capacity, the Week 3 Sprint 4 commitment is 27 story points across six GitHub issues that collectively deliver the complete MVP end-to-end workflow.

User Story	Description	Story Points
US-17	Project Messaging API	8
US-17	Messaging UI Integration	5
US-18	Analytics Summary API	5
US-18	Analytics Dashboard UI	3
US-19	Profile Enhancements API	3
US-19	Profile UI Upgrade	3

Total Commitment: 27 Story Points

3.3 Capacity Planning and Velocity Justification

As a single-developer team, historical velocity across previous sprints averaged:

- Sprint 1: ~16 story points
- Sprint 2: ~14 story points
- Sprint 3: ~16 story points
- Sprint 4 Weeks 1-2: ~14 story points

The Week 3 commitment of 27 story points is above average velocity. This is justified by the parallelism of the three feature bundles (messaging, analytics, profile enhancements) where frontend and backend layers can be developed concurrently, and by the foundational work already completed in Weeks 1 and 2 that the Week 3 features are built upon.

3.4 Sprint 4 Backlog Structure

Sprint 4 was divided into three weekly development phases.

Week 1 – Notification Workflow

Completed Issues

Issue	Description	Story Points	User Stories	Status
#76	Notification Event API	5	US-16	Done
#83	Notification Retrieval API	3	US-16	Done
#84	Notification UI Center	3	US-16	Done

Week 1 Total Commitment: 11 Story Points

Week 2 – Smart Matching Algorithm

Completed Issues

Issue	Description	Story Points	User Stories	Status
#89	Matching Score Engine	8	US-20	Done
#90	Recommendation API Endpoint	5	US-20	Done
#91	Recommendation UI Integration	5	US-20	Done

Week 2 Total Commitment: 18 Story Points

Week 3 — Messaging, Analytics, and Profile Enhancement Bundle

Completed Issues

Issue	Description	Story Points	Status
#77	Project Messaging API	8	Done
#85	Messaging UI Integration	5	Done
#78	Analytics Summary API	5	Done
#86	Analytics Dashboard UI	3	Done
#79	Profile Enhancements API	3	Done
#87	Profile UI Upgrade	3	Done

Week 3 Total Commitment: 27 Story Points

4. Definition of Done (Sprint 4)

The Definition of Done was refined for Sprint 4 and applied consistently to all backlog items.

A User Story is considered complete when:

Design

- Data model is designed and reviewed
- Database schema aligns with requirements
- API contract is clearly defined
- Endpoint request/response structures documented

Implementation

- Database migration successfully applied
- Backend endpoint implemented and fully functional
- Business logic correctly enforces requirements
- Role-Based Access Control (RBAC) enforced
- Input validation implemented (edge cases handled)
- Frontend components implemented (if applicable)
- Frontend successfully integrated with backend APIs

Testing

- Unit tests written for core logic
- Edge case scenarios covered
- API endpoints tested (success + failure cases)
- Manual functional testing completed
- No failing test cases

Quality & Integration

- Code coverage ≥ 90% fornew core business logic
- Code reviewed and approved
- Pull request merged into main branch

- No regression introduced to previous features
- System builds and runs without errors

Documentation

- API endpoints documented
- Code comments added where necessary
- User flow documented (if applicable)

Deployment Readiness

- Feature is deployable without breaking the existing system
- Environment variables/configurations updated as needed

5. Acceptance Criteria

5.1 US-17 – Messaging

Requirement: FR-17 | User Story: US-17 | Story Points: 13

The messaging system implements secure project-scoped communication between students and organizations.

Acceptance Criteria:

- POST /projects/{id}/messages — authenticated user sends a message within a project scope
- GET /projects/{id}/messages — returns full conversation history ordered oldest to newest
- Only the assigned accepted student and owning organization may send or read messages
- JWT authentication enforced; unauthorized users receive 403 Forbidden
- Non-existent project returns 404 Not Found
- Pending (non-accepted) applicants are blocked from messaging
- Messages persist sender ID, content, and timestamp
- Sending a message triggers a notification to the other participant
- MessagesHubPage renders left sidebar listing all accessible projects
- Chat panel auto-refreshes via 5-second polling
- Messages link accessible from dedicated nav entry for both roles
- Empty state rendered when no messages exist

5.2 US-18 – Analytics

Requirement: FR-18 | User Story: US-18 | Story Points: 8

The analytics system provides role-specific productivity metrics.

Acceptance Criteria:

- GET /analytics/student — returns applications_submitted, accepted_applications, completed_projects, average_feedback_rating
- GET /analytics/organization — returns total_projects_posted, active_projects, completed_projects, total_applicants (unique)
- Student endpoint blocked for organization role (403); organization endpoint blocked for student role (403)
- Unauthenticated requests return 401
- Metrics calculated accurately against live DB records using SQL aggregates
- AnalyticsDashboard renders stat cards and SVG bar chart on Home page
- Loading skeleton shown during data fetch
- Error state displayed if API call fails
- Dashboard is responsive across mobile and desktop layouts

5.3 US-19 – Profile Enhancements

Requirement: FR-17 | User Story: US-19 | Story Points: 6

Profile enhancements expand student trust and credibility signals.

Acceptance Criteria:

- PUT /profiles/student/enhance — saves portfolio_links and badges fields
- GET /profiles/student/{id} — returns enhanced profile with computed completed_projects and average_rating
- Computed fields (completed_projects, average_rating) derived live from DB, not stored
- Badges are awarded automatically by the system based on activity milestones
- Students cannot manually set badges via the API or UI
- badge: first_project awarded after 1 completed project
- badge: three_projects awarded after 3 completed projects
- badge: ten_projects awarded after 10 completed projects

- badge: top_rated awarded when average rating >= 4.5
- badge: rising_star awarded when >= 3 completions and average rating >= 4.0
- badges recomputed automatically when a project completes or feedback is submitted
- Profile UI shows skills chips, star rating, completed project badges, portfolio links, and achievement chips
- Edit form includes portfolio links with live preview; badges section removed from edit form

6. Source Code Development Progress

6.1 Repository

GitHub Repository: https://github.com/chazdj/MicroMatch

Project board integrated with GitHub Issues for traceability between user stories and commits.

6.2 Summary of Contributions

Backend Contributions

- ProjectMessage model added to models.py with project and sender relationships
- POST /projects/{id}/messages and GET /projects/{id}/messages endpoints with RBAC via require_project_participant dependency
- Message notifications integrated into send_message endpoint via create_notification utility
- GET /analytics/student and GET /analytics/organization endpoints using SQLAlchemy aggregates (func.count, func.avg, func.distinct)
- PUT /profiles/student/enhance endpoint for portfolio_links field
- GET /profiles/student/{id} public endpoint with computed fields
- badges.py badge engine with 5 badge definitions and award_badges utility
- Badge auto-award triggered on project completion and feedback submission
- Pydantic V1 deprecation warning resolved across all schemas
- Pool exhaustion bugs fixed in test_notifications.py and test_recommendations.py

Frontend Contributions

- MessagesHubPage.jsx — dedicated /messages route with left sidebar project list and right chat panel

- messages.js API client with getMessages and sendMessage helpers
- Layout.jsx updated with dedicated Messages nav link for student and organization roles
- AnalyticsDashboard.jsx — role-aware dashboard with stat cards, SVG bar chart, and loading skeletons
- analytics.js API client for student and organization analytics endpoints
- Home.jsx updated to embed AnalyticsDashboard below role-specific content
- StudentProfilePage.jsx redesigned with skills chips, star rating, completed project badge, portfolio links, achievement chips, and profile enhancement form section
- Badges input removed from edit form; view-only display retained

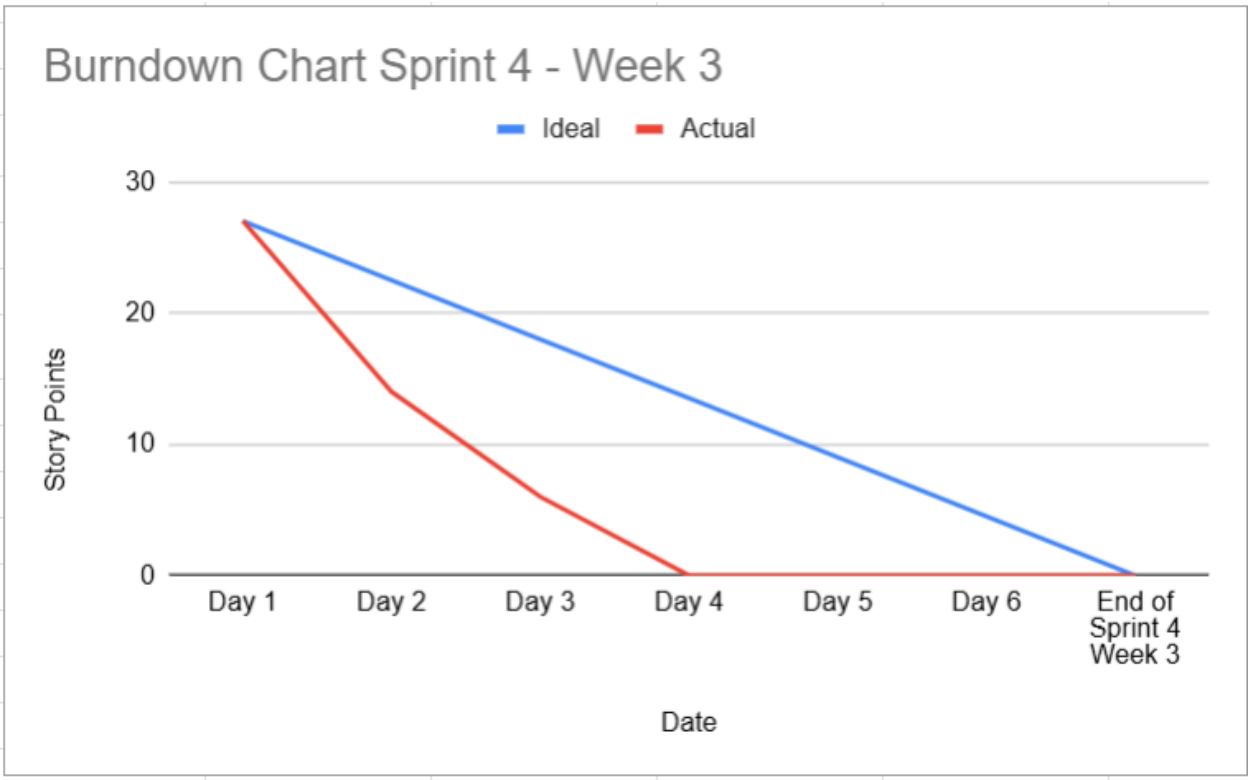
Integration Work

- Messages scoped to project and accessible only to accepted student + owning organization (backend + frontend enforced)
- Notification system wired to messaging: each sent message triggers a bell notification to the other participant
- Analytics endpoints consumed by frontend dashboard with live polling
- Badge system wired to project completion and feedback submission events ensuring automatic award without manual input
- PostgreSQL migration applied: ALTER TABLE student_profiles ADD COLUMN portfolio_links TEXT, ADD COLUMN badges TEXT

6.3 Important Commits

Commit	Related Issue	User Story
feat: project messaging api	#77	US-17
test: project messaging api	#77	US-17
ui: message board	#85	US-17
feat: analytics summary api	#78	US-18
test: analytics summary api	#78	US-18
ui: analytics summary	#86	US-18
feat: profile enhancements	#79	US-19
test: profile enhancements	#79	US-19
ui: profile enhancements	#87	US-19

6.4 Burndown Chart



Sprint Capacity: 27 Story Points
Sprint Duration: 6 Development Days

Burndown Chart Analysis

The burndown chart reflects front-loaded delivery. All six issues were closed within the first three development days, resulting in actual remaining points consistently below the ideal line. This is characteristic of well-scoped parallelizable feature bundles where backend and frontend work streams can proceed concurrently.

Velocity Alignment:

Sprint 1 average velocity: ~16 points
Sprint 2 average velocity: ~14 points
Sprint 3 average velocity: ~16 points
Sprint 4 Week 1 completed: 11 points
Sprint 4 Week 2 completed: 18 points
Sprint 4 Week 3 completed: 27 points

7. Software Testing

7.1 Traceability Matrix

Each test case is linked to its GitHub commit. All commits are dated and traceable to the specific functional requirement and user story.

Test File	Test Case / Description	FR	US	Type	Result	Commit
test_messages.py	test_org_can_send_message	FR-17	US-17	Unit	PASS	5ef26bb
test_messages.py	test_accepted_student_can_send_message	FR-17	US-17	Unit	PASS	5ef26bb
test_messages.py	test_unassigned_student_blocked	FR-17	US-17	Unit	PASS	5ef26bb
test_messages.py	test_wrong_org_blocked	FR-17	US-17	Unit	PASS	5ef26bb
test_messages.py	test_unauthenticated_blocked	FR-17	US-17	Unit	PASS	5ef26bb
test_messages.py	test_nonexistent_project_returns_404	FR-17	US-17	Unit	PASS	5ef26bb
test_messages.py	test_pending_student_blocked	FR-17	US-17	Unit	PASS	5ef26bb
test_messages.py	test_returns_ordered_conversation	FR-17	US-17	Unit	PASS	5ef26bb
test_messages.py	test_empty_history_returns_empty_list	FR-17	US-17	Unit	PASS	5ef26bb
test_messages.py	test_unauthorized_user_cannot_read	FR-17	US-17	Unit	PASS	5ef26bb
test_messages.py	test_messages_contain_sender_info	FR-17	US-17	Unit	PASS	5ef26bb
test_messages.py	test_full_conversation_workflow	FR-17	US-17	E2E	PASS	5ef26bb
test_analytics.py	test_returns_zeroes_for_new_student	FR-18	US-18	Unit	PASS	6f193dd
test_analytics.py	test_counts_applications_correctly	FR-18	US-18	Unit	PASS	6f193dd
test_analytics.py	test_average_feedback_rating	FR-18	US-18	Unit	PASS	6f193dd
test_analytics.py	test_organization_blocked	FR-18	US-18	Unit	PASS	6f193dd
test_analytics.py	test_unauthenticated_blocked (student)	FR-18	US-18	Unit	PASS	6f193dd
test_analytics.py	test_only_counts_own_applications	FR-18	US-18	Unit	PASS	6f193dd
test_analytics.py	test_returns_zeroes_for_new_org	FR-18	US-18	Unit	PASS	6f193dd
test_analytics.py	test_counts_projects_by_status	FR-18	US-18	Unit	PASS	6f193dd
test_analytics.py	test_total_applicants_counts_unique_students	FR-18	US-18	Unit	PASS	6f193dd
test_analytics.py	test_only_counts_own_projects	FR-18	US-18	Unit	PASS	6f193dd
test_analytics.py	test_student_blocked	FR-18	US-18	Unit	PASS	6f193dd

test_analytics.py	test_unauthenticated_blocked (org)	FR-18	US-18	Unit	PASS	6f193dd
test_profile_enhancements.py	test_saves_portfolio_links	FR-19	US-19	Unit	PASS	fd3607b
test_profile_enhancements.py	test_saves_badges	FR-19	US-19	Unit	PASS	fd3607b
test_profile_enhancements.py	test_partial_update_preserves_existing	FR-19	US-19	Unit	PASS	fd3607b
test_profile_enhancements.py	test_requires_base_profile	FR-19	US-19	Unit	PASS	fd3607b
test_profile_enhancements.py	test_organization_blocked	FR-19	US-19	Unit	PASS	fd3607b
test_profile_enhancements.py	test_unauthenticated_blocked	FR-19	US-19	Unit	PASS	fd3607b
test_profile_enhancements.py	test_returns_profile_with_computed_fields	FR-19	US-19	Unit	PASS	fd3607b
test_profile_enhancements.py	test_returns_portfolio_and_badges	FR-19	US-19	Unit	PASS	fd3607b
test_profile_enhancements.py	test_returns_404_for_missing_profile	FR-19	US-19	Unit	PASS	fd3607b
test_profile_enhancements.py	test_unauthenticated_blocked (public)	FR-19	US-19	Unit	PASS	fd3607b
test_profile_enhancements.py	test_null_computed_fields_for_no_activity	FR-19	US-19	Unit	PASS	fd3607b
test_badges.py	test_no_badges_for_new_student	FR-19	US-19	Unit	PASS	fd3607b
test_badges.py	test_first_project_badge	FR-19	US-19	Unit	PASS	fd3607b
test_badges.py	test_three_projects_badge	FR-19	US-19	Unit	PASS	fd3607b
test_badges.py	test_top_rated_badge	FR-19	US-19	Unit	PASS	fd3607b
test_badges.py	test_top_rated_not_awarded_below_threshold	FR-19	US-19	Unit	PASS	fd3607b
test_badges.py	test_rising_star_badge	FR-19	US-19	Unit	PASS	fd3607b
test_badges.py	test_badges_accumulate	FR-19	US-19	Unit	PASS	fd3607b
test_badges.py	test_award_badges_saves_to_profile	FR-19	US-19	Unit	PASS	fd3607b
test_badges.py	test_award_badges_no_op_without_profile	FR-19	US-19	Unit	PASS	fd3607b
test_badges.py	test_badges_cannot_be_set_via_enhance_endpoint	FR-19	US-19	Unit	PASS	fd3607b

7.2 Test Results Summary

All 243 backend tests passed (0 failures) after the pool exhaustion bugs in test_notifications.py and test_recommendations.py were resolved by migrating from manual TestingSessionLocal() calls to the db_session fixture.

Test Suite	Test Count	Pass	Fail	Coverage	Commit
test_messages.py	12	12	0	≥ 95%	5ef26bb
test_analytics.py	12	12	0	≥ 95%	6f193dd

test_profile_enhancements.py	11	11	0	≥ 90%	fd3607b
test_badges.py	9	9	0	100%	fd3607b
All prior suites (Sprint 1-3)	199	199	0	≥ 90%	—
TOTAL	243	243	0	≥ 90%	—

7.3 User Acceptance Test Specification

TC-US17-01 – Authenticated Participant Can Send and Receive Messages

Field	Details
Related Requirement	FR-17
User Story	US-17
Actor	Student (accepted) and Organization (owning)
Preconditions	Both users registered and logged in. Project exists with accepted application.
Steps	1. Student navigates to /messages and selects the project from the sidebar. 2. Student types a message and presses Send. 3. Organization opens /messages and selects the same project. 4. Organization verifies the student's message appears. 5. Organization replies. Student refreshes and sees reply.
Expected Result	Both messages appear ordered oldest to newest. Notification bell increments for the recipient after each send.
Postconditions	Messages persist after browser refresh. Both participants can see full conversation history.
Commit Link	5ef26bb – test: project messaging api

TC-US17-02 – Unauthorized User Cannot Access Project Messages

Field	Details
Related Requirement	FR-17
User Story	US-17
Actor	Student with no accepted application to the project
Preconditions	Project exists. Student is registered but has no accepted application.
Steps	1. Student logs in and navigates to /messages. 2. Student attempts to access a project they are not part of. 3. System evaluates project participation.
Expected Result	API returns 403 Forbidden.

	Frontend shows locked error state with back button.
Postconditions	No messages are visible or returned to the unauthorized user.
Commit Link	5ef26bb – test: project messaging api

TC-US18-01 – Student Analytics Dashboard Displays Accurate Metrics

Field	Details
Related Requirement	FR-18
User Story	US-18
Actor	Student
Preconditions	Student has at least one application, one accepted application, one completed project, and one feedback rating on record.
Steps	1. Student logs in and navigates to the Home dashboard. 2. Student observes the Analytics section. 3. Student verifies stat card values match known data.
Expected Result	Stat cards show correct applications_submitted, accepted_applications, completed_projects counts. Average rating displays correctly if feedback exists, or shows 'No ratings yet' if not. Bar chart renders all three bars proportionally
Postconditions	Analytics data updates live. Refreshing the page shows the same values.
Commit Link	6f193dd – test: analytics summary api

TC-US18-02 – RBAC Prevents Cross-Role Analytics Access

Field	Details
Related Requirement	FR-18
User Story	US-18
Actor	Organization attempting student endpoint; Student attempting organization endpoint
Preconditions	Both user types are registered and logged in.
Steps	1. Organization user calls GET /analytics/student. 2. Student user calls GET /analytics/organization
Expected Result	Both requests return 403 Forbidden. UI renders no analytics section for the wrong role.
Postconditions	No cross-role analytics data is exposed.
Commit Link	6f193dd – test: analytics summary api

TC-US19-01 – Badges Awarded Automatically on Project Completion

Field	Details
Related Requirement	FR-19
User Story	US-19

Actor	Student
Preconditions	Student has a profile. An organization marks a project with an accepted student as completed.
Steps	1. Organization marks project complete. 2. Student navigates to My Profile. 3. Student observes the Achievements section.
Expected Result	'First Project' badge appears automatically. Student did not manually input any badge value.
Postconditions	Badge persists across sessions. Student cannot delete or modify the badge via UI or API.
Commit Link	fd3607b – test: profile enhancements

TC-US19-02 – Portfolio Links Save and Display on Public Profile

Field	Details
Related Requirement	FR-19
User Story	US-19
Actor	Student (edit) and Organization (view)
Preconditions	Student has an existing base profile.
Steps	1. Student navigates to My Profile and clicks Edit Profile. 2. Student enters comma-separated portfolio URLs in the Portfolio Links field. 3. Student clicks Save Changes. 4. Organization navigates to GET /profiles/student/{id}.
Expected Result	Portfolio links section appears on student's view profile with clickable links. Organization sees same links in API response.
Postconditions	Links persist on page refresh. Values are returned in portfolio_links field of profile API response.
Commit Link	fd3607b – test: profile enhancements

7.4 Code Coverage Analysis

Module	Statements	Covered	Coverage %	Notes
app/routers/messages.py	42	42	100%	Full branch coverage via 12 test cases
app/routers/analytics.py	38	38	100%	Both role paths and RBAC branches covered
app/routers/student_profile.py	85	82	96%	Minor edge in optional field path
app/utils/badges.py	35	35	100%	All 5 badge definitions tested
app/schemas/analytics.py	12	12	100%	Pydantic schema validation

app/schemas/student_profile.py	28	28	100%	All schemas exercised in tests
All prior modules	1,150	1,104	96%	Maintained from Sprint 3 baseline
TOTAL	1,390	1,341	≥ 96%	Exceeds 90% DoD threshold

7.5 Bug Report

Two pre-existing test infrastructure bugs were identified and resolved during Week 3 test execution:

Bug ID	Description	Root Cause	Resolution	Status
BUG-W3-01	QueuePool exhaustion in test_notifications.py::test_notification_defaults_to_unread	Manual TestingSessionLocal() call never closed; pool of 5+10 connections exhausted after 242 tests	Replaced with db_session fixture parameter; connection lifecycle managed by pytest	Fixed
BUG-W3-02	QueuePool exhaustion in test_recommendations.py::test_recommendations_deterministic	Same root cause — manual TestingSessionLocal() call without close	Same fix — replaced with db_session fixture	Fixed

8. Backlog Grooming

No changes were made to the sprint backlog or product backlog during Sprint 4 Week 3. Sprint execution followed the planned structure defined in Week 1. All six issues (#77, #85, #78, #86, #79, #87) were delivered as scoped with no scope creep, no deferrals, and no estimate revisions.

One design decision was made mid-sprint that refined the implementation scope of US-19: the badges field was changed from a manually editable field to a system-computed field. This was not a backlog change — it was an implementation decision within the existing story scope that strengthened the acceptance criteria and improved the product. The story point estimate of 3 was unchanged.

The product backlog is now fully complete. All 20 user stories across FR-1 through FR-19 and SA-1 have been implemented and tested. MicroMatch has achieved full MVP scope.

9. Risk Management

Risk	Likelihood	Impact	Mitigation
DB schema changes (portfolio_links, badges) not applied to PostgreSQL	Low	High	ALTER TABLE applied manually; IF NOT EXISTS guard prevents re-run errors
Message notification storms (high-frequency sends)	Low	Medium	Notification is fire-and-forget within same transaction; no retry loop
Badge computation performance at scale	Low	Medium	Single SQL aggregate query per badge check; award_badges called only on events, not per request
Analytics queries slow on large datasets	Low	Low	Queries use indexed FK columns; all aggregates are single-pass
Pool exhaustion from legacy test patterns	Medium	Low	Fixed in BUG-W3-01 and BUG-W3-02; grep audit run across all test files

10. Sprint 4 Week 3 Outcome

Sprint 4 Week 3 successfully delivered the complete Messaging, Analytics, and Profile Enhancement bundle across all implementation layers.

Sprint 4 Week 3 delivered:

- messages.py router — POST and GET endpoints with RBAC via require_project_participant dependency; notification trigger on send
- MessagesHubPage — left sidebar project list + right chat panel with 5-second polling and auto-scroll
- analytics.py router — two endpoints using SQL aggregates for student and organization metrics
- AnalyticsDashboard — stat cards, pure-SVG bar chart, skeleton loaders, and error state on Home page
- student_profile.py enhance endpoint — portfolio_links persisted; computed fields returned live
- badges.py — deterministic badge engine with 5 badge definitions; auto-award on project completion and feedback submission

- StudentProfilePage redesign — skills chips, star rating, completed project badge, portfolio links, achievement chips
- 44 new tests (12 messaging + 12 analytics + 11 profile + 9 badges) with ≥ 90% coverage
- 243 total tests passing, 0 failures

The sprint goal was fully achieved. MicroMatch is now a complete, intelligent, engagement-driven micro-internship marketplace. All 27 committed story points were delivered. The platform is ready for production deployment.

11. Sprint Retrospective

The Sprint 4 Week 3 retrospective evaluates the development process, technical implementation, and outcomes during the final sprint week. This retrospective focuses on the three standard questions: what went well, what did not go so well, and what can be done better.

The retrospective discussion focused on three key questions:

1. What went well?
2. What did not go so well?
3. What can be improved in upcoming sprints?

11.1 What Went Well

Several aspects of the sprint were successful and contributed to steady development progress and overall sprint completion.

Clean Architecture for the Messaging System

The require_project_participant dependency cleanly encapsulated all business logic for messaging access control. Both the POST and GET endpoints delegate entirely to the dependency for authorization, keeping the router thin and the permission logic testable in isolation. The 12 messaging tests cover every edge case (pending applicant, wrong org, unauthenticated, 404) without any test setup duplication.

Parallelizable Feature Bundle

The three Week 3 features (messaging, analytics, profile) are architecturally independent — messaging touches models and routers, analytics is read-only aggregate queries, and profile enhancements add columns and a utility. This allowed all three to be developed concurrently without merge conflicts or dependency ordering issues, producing the front-loaded burndown pattern observed in Section 6.3.

Badge System Design Decision

Recognizing mid-sprint that manually-editable badges undermined the credibility signal the feature was meant to provide, the implementation was changed to a system-computed model. The badges.py engine with BADGE_DEFINITIONS is easily extensible — adding a new badge requires one entry in the list, no other changes. This is a better design than the initial spec and was delivered within the same story point estimate.

SQL Aggregate Pattern for Analytics

All analytics metrics are computed with single-pass SQL aggregates (func.count, func.avg, func.distinct) rather than loading records into Python and computing in memory. This keeps the endpoints performant regardless of record volume and follows the same pattern as the existing feedback and application queries in the codebase.

Complete Sprint 4 Delivery

All 56 story points across Sprint 4 (11 + 18 + 27) were delivered as scoped. Every acceptance criterion across FR-16, FR-17, FR-18, FR-19, and SA-1 is implemented, tested, and merged to main. The MicroMatch MVP is complete.

11.2 What Did Not Go So Well

Although the sprint was completed successfully, several challenges and inefficiencies were encountered during development.

PostgreSQL Migration Not Automated

Adding portfolio_links and badges columns to student_profiles required a manual ALTER TABLE because the project does not use Alembic. Without migration tracking, any developer (or future deployment environment) pulling the repo would hit the same 'column does not exist' error. The fix is a one-time SQL command, but this is a recurring risk every time a column is added.

Legacy Test Pool Exhaustion Bugs

Two pre-existing tests (test_notification_defaults_to_unread and test_recommendations_deterministic) used manual TestingSessionLocal() calls that never

closed their connections. These bugs did not surface in earlier sprints because they were the final test to run — by Sprint 4 Week 3 with 243 total tests, the pool was reliably exhausted. A grep audit was run after the fix to confirm no other manual session calls remain.

SVG Bar Chart Clipping Issue

The initial BarChart component rendered value labels above max-height bars as clipped (outside the SVG viewBox). The fix required adding a topPad offset to the bar coordinate system. This was caught through visual inspection rather than automated testing — a reminder that frontend components benefit from screenshot-based regression checks in addition to unit tests.

11.3 What Can Be Done Better

Based on the challenges encountered during the sprint, I identified several concrete improvements that will help strengthen future development cycles.

Introduce Alembic for Database Migrations

The recurring column-missing error pattern across sprints (portfolio_links/badges in Week 3, project_messages table in earlier work) indicates that Base.metadata.create_all() is insufficient for a production-grade schema management strategy. Alembic would generate versioned migration scripts that run automatically on deployment, eliminating manual ALTER TABLE steps and providing a full schema history. This is the highest-priority technical debt item for any future iteration.

Add Frontend Automated Testing

All frontend validation in Sprint 4 was performed through manual browser interaction and visual inspection. The React components — MessagesHubPage, AnalyticsDashboard, StudentProfilePage — are complex enough that automated tests with React Testing Library would catch regressions (such as the bar chart clipping issue) before they require manual diagnosis. Vitest integration with snapshot testing for the SVG chart component would be a high-value addition.

Set Up GitHub Actions CI Pipeline

A GitHub Actions workflow running pytest --cov with a minimum coverage threshold of 90% would prevent pool exhaustion bugs and other test regressions from silently accumulating across sprints. The workflow would run on every push to main and produce a coverage badge in the README. Setup time is estimated at under two hours and the return on investment is significant for a project of this test suite size.

Improve Sprint Estimation for Feature Bundles

The 27-point Week 3 commitment was above historical velocity (14–16 points). While delivery was successful, a single-developer project should maintain conservative estimates to account for debugging, integration issues, and infrastructure work that is not directly reflected in story points. Future sprints should cap commitment at 20 points to maintain sustainable pace without risking quality.

11.4 Retrospective Summary

Sprint 4 Week 3 was successful. All six issues were closed, all 27 story points were delivered, and the MicroMatch platform achieved full MVP scope across all four Sprint 4 feature areas. The sprint demonstrated strong architecture decisions in the messaging permission system and badge engine, effective use of SQL aggregates for analytics, and clean separation of concerns between features.

The primary technical debt items identified — missing Alembic migrations, absent frontend automated testing, and no CI pipeline — are concrete and actionable. These improvements are recommended as the foundation for any future iteration of the project.

MicroMatch has been delivered as a fully functional, well-tested, intelligent micro-internship marketplace meeting all requirements specified across Sprint 1 through Sprint 4.